

Efficient Skill Grounding via Code Refactoring with Small Language Models

RECENT: 利用小语言模型局部代码重构实现高效技能适配

Sera Choi¹, Wonje Choi¹, Saehun Chun¹, Daehee Lee¹, Jooyoung Kim¹,
Chaeun Lee², Honguk Woo¹

¹ Department of Computer Science and Engineering, Sungkyunkwan University

² Department of Systems Management Engineering, Sungkyunkwan University

ICML 2026

汇报人: 冯远坤



성균관대학교
SUNGKYUNKWAN UNIVERSITY



ICML 2026
INTERNATIONAL CONFERENCE
ON MACHINE LEARNING

作者

研究方向:

大语言模型

近期工作:

- Efficient Skill Grounding via Code Refactoring with Small Language Models **ICML' 26**
提出 RECENT 框架，在部署阶段利用 ontology guidance 与 实现高效、可靠的跨机器人 skill grounding。
- FCGRAFT: Functional Cache Grafting for LLM-Generated **ICML' 26**
通过复用与嫁接历史函数缓存，加速具身控制代码生成，并提升生成代码的稳定性



Sera Choi

Ph.D. Student

Memory Team, CSI-Agent Group

Sungkyunkwan University

<https://csiagentgroup.com/people/>

提纲

1

研究背景

2

系统设计

3

实验评估

4

总 结

提纲

1

研究背景

2

系统设计

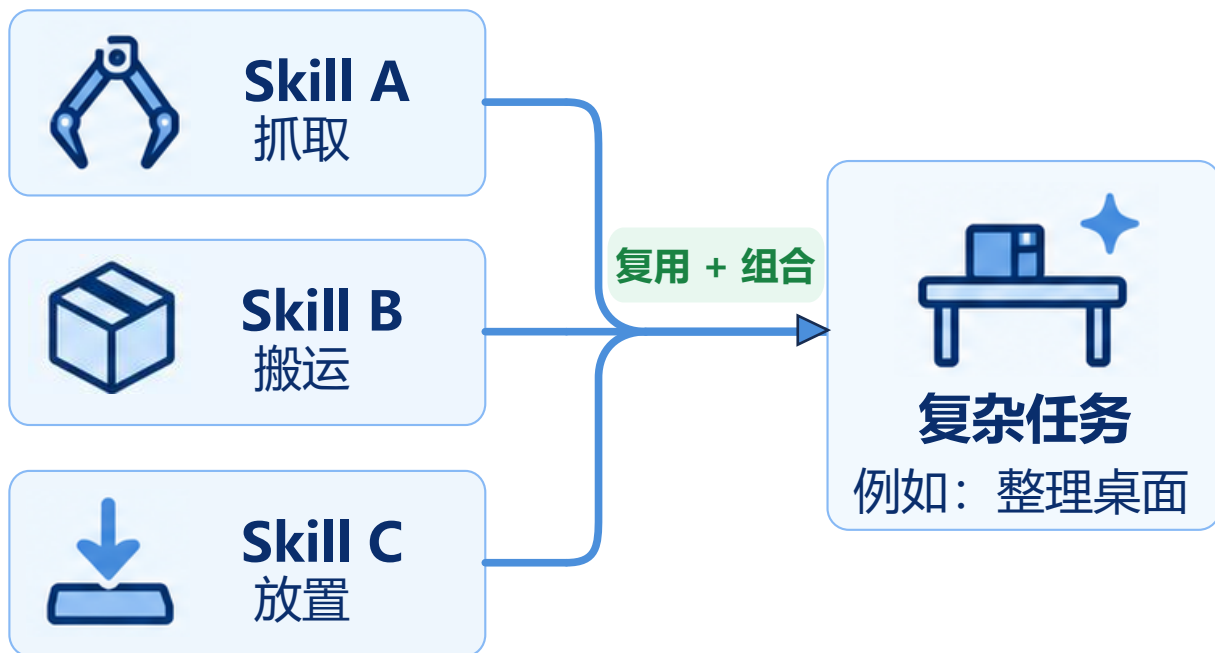
3

实验评估

4

总 结

具身智能中的技能复用需求



目标：技能应尽可能复用

- 不希望每个新任务都重新训练一套控制策略
- 希望复用并组合已有 skill 完成复杂任务
- 提升任务求解效率与泛化能力

具身智能需要可复用、可组合的技能，而不是每次从零开始学习。

已有技能的复用与组合，是提升具身智能机器人任务效率的重要途径。

研究背景

机器人本体与环境差异导致已有技能无法直接复用

1 机器人本体差异

不同机器人的运动学结构、末端执行器和可用 API 存在差异

运动学结构



Franka Panda



UR5



Sawyer

...

将已有 Skill 适配到目标机器人与当前环境，使其能够正确执行的过程，称为 Skill Grounding (技能适配)

已有 Skill → 新机器人 / 新环境 → Skill Grounding → 可执行 Skill

可用 API

Franka API
close_gripper()
move_to_pose()
...

UR5 API
set_gripper(width)
movej()
...

2 环境状态差异

环境中的动态变化、物体状态和场景约束也会导致原技能失效

物体姿态变化



场景障碍物



接触与物理特性



原环境：可以直接
从侧面抓取



新环境：侧面被障碍物
挡住，原抓取动作失效

13

研究背景

早期的 Skill Grounding: 重新训练 / 微调 Neural Policy

1 方法: 基于神经网络策略的技能适配

将每个技能表示为一个参数化的神经网络策略, 在新的机器人或环境中重新训练或微调, 使其能够成功执行。



2 优点

- ✓ 在特定机器人和环境中可以获得较高的执行性能
- ✓ 端到端学习, 能够适应复杂的动力学和接触

3 不足

- ✗ 需要额外的训练数据
- ✗ 重新训练或微调的计算和时间成本高
- ✗ 技能的功能语义与执行细节耦合在策略参数中, 难以只修改部分执行方式

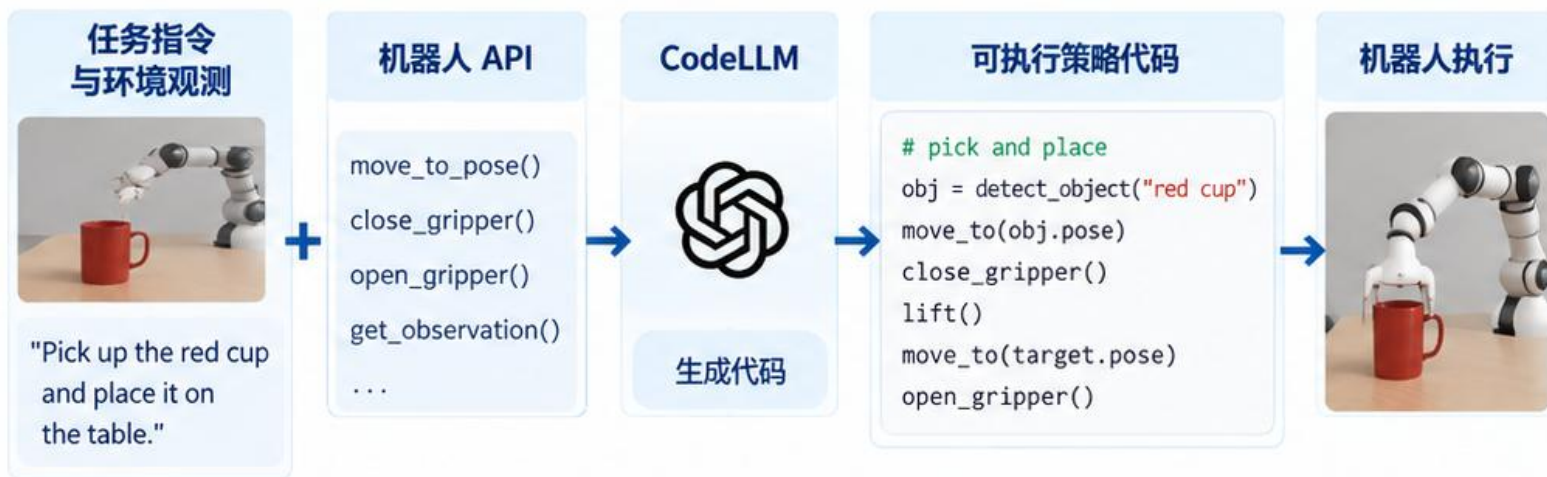
! 当机器人或环境发生变化时, 重新训练整套神经网络成本较高, 难以实现高效的部署适配。

研究背景

Code-as-Policies (CaP) : 通过 LLM 生成机器人控制代码

1 方法：通过代码生成进行 Skill Grounding

使用LLM根据任务指令、当前环境观测和机器人 API 生成可执行的机器人控制代码



2 相比神经网络策略的优势



不需要重新训练策略



- 修改代码即可适配新的机器人或环境
- 无需收集数据和重新优化神经网络



代码结构显式，可理解可修改



- 控制逻辑、API 调用和参数清晰可见
- 便于查看、调试、复用和人工修改



Code-as-Policies 将skill从隐式的参数化策略转为显式的可执行代码，因此skill迁移时只需要修改对应的代码即可

研究背景

CaP的不足①：局部不兼容，却需要重新生成完整程序

1 现有做法：执行失败后重生成整个 Skill

当生成的代码在新机器人或新环境中执行失败时，CaP 通常将错误信息反馈给 LLM，并重新生成完整程序，而不是只修改发生不兼容的局部代码。

原始 Skill 代码 (在新环境中执行)

```
detect_object("red cup")
move_to(obj.pose)
close_gripper()
lift()
move_to(target.pose)
open_gripper()
```

真正不兼容的可能只是一小段
execution binding

执行失败 + Error Feedback



! Execution Error:
close_gripper() failed:
gripper type not supported

重新生成整个 Skill

```
detect_object("red cup")
move_to(obj.pose)
grasp_with_vacuum()
lift()
move_to(target.pose)
release_vacuum()
```

整份程序被重新生成

2 主要问题

开销变高



- 即使只有少量代码需要修改，也需要重新生成整个程序
- 生成 token 更多、推理 latency 更高

可能破坏原本正确的代码



- 重新生成大量本来正确的代码
- 原本正确的程序结构也可能被改错



CaP 虽然避免了重新训练 Neural Policy，但仍存在“局部错误、全量重写”的问题：既增加开销，也可能把原来正确的代码改错。

研究背景

CaP的不足②：部署阶段缺少大型 LLM，SLM 难以完成完整 Skill 重生成

1 传统 CaP 对模型的要求

在执行失败后，模型不仅要理解错误信息，还要重新组织完整程序，因此通常依赖强大的云端 LLM。



2 部署阶段的现实约束



为什么 SLM 难以胜任 Full Regeneration?

- 部署时通常只能使用更小、更便宜的模型 (SLM)
- SLM 的推理有限，难以独立完成“诊断 + 定位 + 修改决策 + 完整代码生成”
- 如果仍依赖从头重写代码，SLM 的负担过重，可靠性不足



因此，端侧真实部署时，依靠SLM从头生成代码，完成Skill的环境适配不够现实

研究背景

核心问题：没有云端 LLM，SLM 如何完成 Skill Grounding?

1 真实部署阶段



2 为什么现有 CaP + SLM 很难?



论文要解决的问题

在没有云端 LLM 的部署阶段，如何仅依赖 SLM，将已有 Skill 高效、可靠地适配到新的机器人和环境？

提纲

1

研究背景

2

系统设计

3

实验评估

4

总 结

系统设计

RECENT 系统整体架构

 **Panda robot**
> chop vegetables

Functional requirements


Vegetables sliced into pieces

Robot embodiment


Robot specs & primitive APIs

Semantic relations


Requires / Provides / Substitutable

Determined conflicts ⚠️

 `panda_move(pose)`  **API mismatch** ✓ ✗

(e.g., sensing **visibility**, contact **stability**, or object **properties**)

Ontology-based compatibility analysis



Target robot embodiment



Undetermined warnings ⚠️


from the side? or from above?

(e.g., sensory **feedback**, contact **outcomes**, or object **states**)

RECENT

MainTaskCode

```
[1] def run_skill_chop(env, robot, ...):  
    def move_fn(pose, **kw):  
        - robot.panda_move(pose)  
        + robot.my_ur5_move_to(target_pose=pose, **kw)  
        return None  
        knife_object = Shape('knife')  
        knife_position = knife_object.get_position()  
        move_returns = move_fn(knife_position)  
        ...  
[2]  
[3] knife_sizes, _ = get_bbox_sizes(knife_object)  
    l = float(np.max(knife_sizes)) / 4.0  
    ...  
[4] knife_handle_pos = knife_position.copy()  
    ...  
[5] knife_handle_pos[2] += 1  
    obs, reward, done = pick(env, task,  
        target_pos=knife_handle_pos,  
        approach_distance=0.05,  
        approach_axis='y')  
[6] knife_pos = knife_handle_pos.copy()  
    knife_pos[2] += 1
```

RECENT

MainTaskCode

```
[1] def run_skill_chop(env, robot, ...):  
    def move_fn(pose, **kw):  
        robot.my_ur5_move_to(  
            ...  
[2]   
[3] Environment update  
[4] knife_handle_pos[2] += 1  
    obs, reward, done = pick(e  
        task, target_pos=knife_han  
        - approach_distance=0.05,  
        - approach_axis='y')  
        + approach_distance=0.2,  
        + approach_axis='z')  
[5] knife_pos = knife_handle_pos.copy()  
    knife_pos[2] += 1  
    ...
```


Running...
Observation
Unit tests on ontology
Edit

(i) Skill repository

(ii) Ontology-based reasoning

(iii) In-situ adaptation

离线技能库： 存储已验证的 skill 代码，以及 skill 功能需求、机器人能力与对应的关系。

本体推理： 识别将 skill 移植到目标机器人后可能存在的两种冲突。

在线适配： 结合环境反馈与单元测试，只局部修补不兼容代码，而不是完整重写。

系统设计

1. Skill Repository: 构建 Skill Ontology

Skill Ontology 是什么?

描述 Skill—Robot—Capability—API 之间关系的结构化知识图谱

1. 识别 Skill 需要什么

3. 若不满足, 给出可替代方案

2. 判断 Target Robot 是否满足

输入 1:
Robot embodiment
specification

- URDF: 关节、运动范围、连杆等
- SRDF: group、语义配置等



机器人说明书

输入 2:
Primitive API
specification

API: `close_gripper()`
Precondition: object is inside gripper region
Effect: object becomes grasped
Capability: parallel-jaw grasp

输入 3:
Reference Skill Code

- 提取 primitive 调用位置
- 分析参数从哪里来
- 分析调用顺序 / 依赖关系

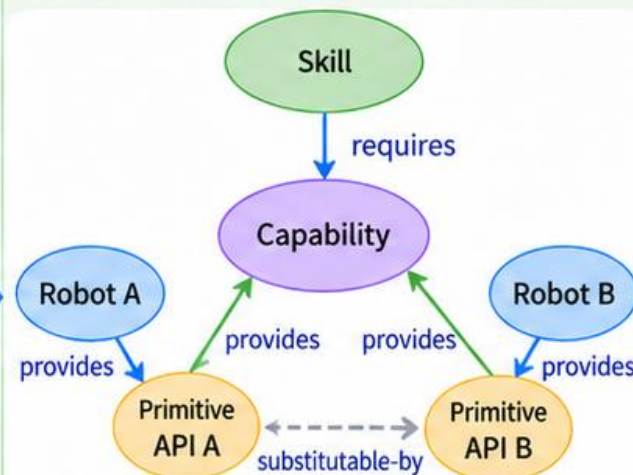
```
1 def pick_and_place():  
2     close_gripper()  
3     move_to(target)  
4     open_gripper()
```

</>


deterministic
compatibility
matching rules

Ontology 自动构建程序

输出: Skill Ontology



- requires: 需要 / 依赖
- provides: 提供 / 具备 / 实现
- - - substitutable-by: 可替代为



后面系统就可以根据 Ontology 判断: 某个 Skill 需要什么能力, 目标机器人能不能满足, 如果不能, 是否存在可以替代的 API

1. Skill Repository: 构建 Skill Repository

1. Skill Ontology



- 功能需求
- preconditions
- expected effects
- Primitive API

完整的结构化知识库

2. Ontology Query

输入: Skill χ + Source Robot r_{src}

$$Q(\chi, r_{src}) = \langle C(\chi), P(\chi), E(\chi), I(\chi, r_{src}) \rangle$$

- C: 需要什么能力 / 约束
- P: 执行前需要满足什么条件
- E: 执行后应该达到什么效果
- I: 在当前机器人上的 primitive API interface

查询出面向源机器人的 skill specification

3. Cloud LLM 生成 Skill Code



```
def pick_and_place():  
    move_to(target)  
    close_gripper()  
    return True
```

根据skill的要求生成对应code实现

4. Source Robot Validation



- ✓ Preconditions 是否满足
- ✓ 执行结果是否符合 expected effects
- ✓ Skill 是否可执行

仅验证通过的 Skill 才可入库

5. Validated Skill Repository



为部署阶段提供可靠的skill



只有验证通过的 **Skill** 才会进入 **Repository**，这样部署阶段面对新的目标机器人时，就可以从一个已经验证过、语义明确的 **Skill** 出发，而不需要让 **SLM** 从头重新生成整个 **Skill**。

系统设计

2. Ontology-based reasoning: 识别skill与机器人的不兼容

给定 Repository 中已验证的 Skill 和目标机器人，系统先进行本体兼容性分析，判断哪些部分可直接复用，哪些部分存在不兼容。



💡 这一阶段最重要的作用就是：先把哪些问题现在就能解决、哪些问题必须留到运行时再处理区分清楚，为后面的局部重构和在线适配做准备。

系统设计

2. Ontology-based reasoning: SLM局部修改已确定冲突 (Determined Conflicts)

对于已确定的冲突，RECENT 不会重新生成整个 Skill，而是在Ontology的指导下，仅修改不兼容的局部代码片段。

1: 确定冲突

! 确定冲突

- API 不匹配
- 执行前即可确定

示例:

```
robot.panda_move(pose)
```

Target Robot: UR5

2: 定位需要修改的代码片段

```
1 def execute():
2     home()
3     robot.panda_move(pose)
4     gripper.close()
5     return True
```

Prefix (保持不变)

可编辑片段 m_i

Suffix (保持不变)

只定位不兼容的局部代码，
不改动其余 Skill 结构。

3: Ontology提供提示

📖 替换提示 κ_i

- 原API:
panda_move(...)
- 替代API:
my_ur5_move_to(...)

目标: 在 Target Robot
上实现相同 skill 功能。

4: SLM 局部修改

FIM 形式的局部代码补全

Prefix + [MASK] + Suffix + 提示 κ_i

SLM

生成结果 (局部替换)

只修改局部 span

```
- robot.panda_move(pose)
+ robot.my_ur5_move_to(target_pose=pose, **kw)
```

❌ 不是完整重写整个 Skill

✅ 而是在 Ontology 指导下做
局部代码重构



核心作用: 完整 Skill 重生成” 转化为 “局部代码补全”，显著降低 SLM 的生成负担。

系统设计

3. In-situ adaptation: 用运行时unit tests检测潜在警告 (Undetermined Warnings)

对于执行前无法确定的问题，RECENT 会将 Undetermined Warnings 转化为 unit tests，并结合当前 observation，在相关代码真正执行前判断是否需要适配。

1: Undetermined Warnings

⚠ Undetermined Warnings

- 是否存在遮挡
- 物体的物理属性改变

执行前只能怀疑，
无法完全确定

2: 转化为 Unit Test

📋 将 warning 写成可检查条件

- 检查 precondition 是否仍满足
- 检查 expected effect 是否仍成立

示例：
“侧面接近是否仍可行？”

每个 warning 对应一个
轻量验证检查

3: 执行中结合 Observation

📷 Observation

- 相机图像
- 接触状态
- 物体状态

📋 Unit Test

- 基于当前 observation 进行检查



检查尚未执行的代码，不是失败后再修。

在目标代码真正执行前发现问题

4: 检查结果

✅ 通过

- 条件满足
- 继续执行原代码

❌ 失败

- warning 被确认
- 下一步：在线局部适配



核心作用：把执行前无法确定的问题延迟到运行时检查，并在必要时为后续在线局部修复提供触发信号。

3. In-situ adaptation: 根据环境反馈进行在线局部修复

当运行时 observation 未通过unit test, RECENT 只对尚未执行的代码片段进行局部修复

1: 触发在线修复

❌ 执行时发现问题

- Unit Test 失败
- warning 被确认
- 需要在线适配

示例:
“侧面接近不可行”

发生在执行过程中

2: 定位待修改代码

```
1 def execute():
2     move_to_pregrasp()
3     # editable span
4     approach_distance = 0.05
5     approach_axis = 'y'
6     grasp()
```

Prefix (已执行 保持不变)

可编辑片段 m

Suffix (未执行 其余结构保持)

只修改尚未执行的代码

3: 结合 Observation 修复

当前 Observation

- 环境更新 / 遮挡变化
- 接触状态 / 物体状态

SLM 局部补全

Prefix + [MASK] + Suffix + Observation

SLM

```
- approach_distance = 0.05
- approach_axis = 'y'
+ approach_distance = 0.2
+ approach_axis = 'z'
```

4: 验证并继续执行

✅ 验证通过, 继续执行

- 修复后Unit Test 通过
- 从当前位置继续执行

❌ 仍然失败

- 若仍失败, 可再次局部 patch



这一阶段的核心是: 不重新生成整个 Skill, 也不重做已经完成的步骤, 而是根据运行时环境反馈, 只修改后续真正受影响的局部代码。

提纲

1

研究背景

2

系统设计

3

实验评估

4

总 结

实验评估

实验设置：在哪些 Skill Grounding 场景下评估？

场景一：Kinematic Variation



Panda
(源机器人)



UR5 Sawyer
(目标机器人)

- 源机器人：Panda
- 目标机器人：UR5 / Sawyer
- 变化：机械臂结构、关节配置、运动学约束

场景二：End-effector Variation



Panda + 平行夹爪
(源机器人)



Robotiq 2F-85 Vacuum gripper
(目标末端执行器)

- 源机器人：Panda + 平行夹爪
- 目标末端执行器：Robotiq 2F-85 / Vacuum gripper
- 变化：夹持机制

评估指标



SR ↑

任务成功率



GC ↑

子目标完成率



GO ↓

Grounding Overhead
(新生成 token / 原 skill token)



EI ↓

执行中断次数



IT ↓

因修复导致的等待时间

仿真主实验：跨机器人 Skill Grounding



实验目的：验证在 SLM 部署约束下，RECENT 能否通过局部代码重构代替完整 Skill 重生成，在不同机器人本体变化下同时保证任务成功、降低 Grounding 开销并减少执行中断。

METHOD	Kinematic Variation					End-effector Variation				
	SR (%) ↑	GC (%) ↑	GO (%) ↓	EI (count ↓)	IT (sec ↓)	SR (%) ↑	GC (%) ↑	GO (%) ↓	EI (count ↓)	IT (sec ↓)
CAP	17.00±2.09	41.11±2.49	102.01±2.86	4.72±0.13	25.65±1.33	21.97±2.54	36.33±7.93	79.31±2.45	3.18±0.38	55.62±0.85
CAP-CODEV-R1	20.00±10.00	36.10±13.72	109.38±33.84	4.13±2.49	17.50±6.65	17.88±6.63	41.03±10.81	99.16±9.47	2.77±0.84	43.77±2.43
SCoT	14.50±6.47	38.38±2.31	107.08±28.69	4.40±1.82	28.86±10.81	15.83±1.86	54.19±1.86	85.35±12.50	1.80±0.22	41.87±9.31
PROGPROMPT	31.50±7.20	53.89±7.07	86.47±5.54	3.28±1.42	30.74±29.83	33.33±11.18	58.16±7.40	65.34±0.77	4.72±0.79	39.32±5.67
ROBOINSPECTOR	41.00±4.59	60.83±6.20	179.40±81.75	1.15±0.45	59.98±33.97	39.17±14.72	64.80±5.41	52.20±10.24	5.07±0.33	31.65±10.37
REPAIRAGENT	42.00±4.11	64.05±2.50	54.17±19.87	1.73±0.14	50.53±17.35	52.17±2.09	73.82±3.04	55.92±28.83	1.12±0.25	84.37±75.24
AGENTLESS	22.50±13.31	42.70±12.42	128.08±1.18	2.19±0.61	58.27±20.82	26.34±13.98	49.88±13.62	74.96±5.44	5.01±0.50	42.44±11.10
RECENT (OURS)	73.00±3.71	81.76±1.07	7.85±3.00	0.72±0.35	1.77±0.90	82.50±1.86	89.84±2.05	2.51±0.08	0.69±0.44	2.34±0.93
CAP-CODEX	67.50±8.48	80.45±4.81	84.29±25.78	1.23±0.51	7.99±4.11	74.85±2.97	87.81±1.74	41.69±8.28	0.60±0.08	10.29±4.33

实验结果：RECENT 在两类场景下均取得最高的 SR 和 GC，最低的GO、IT，执行中断次数也显著更少。

实验结论：在相同 SLM 部署约束下，RECENT 明显优于依赖完整代码重生成的 CaP；通过保留 Skill 语义、仅进行局部修复，可以同时提高任务成功率、降低 Grounding 开销并保持连续执行。

消融实验：为什么 RECENT 有效？



实验目的： 验证 RECENT 中关键设计模块的有效性，分析 Ontology Guidance、Localized Refactoring 对整体性能的贡献，说明为什么 “只做局部代码重构” 能够取得更好的跨机器人 Skill Grounding 效果。

METHODS	SR (% $\uparrow$)	GO (% $\downarrow$)	EI (count $\downarrow$)	IT (sec $\downarrow$)
W/O. ONTOLOGY GUIDANCE	21.67 $\pm$ 7.45	2.35 $\pm$ 1.55	2.57 $\pm$ 0.59	4.14 $\pm$ 2.15
→ W/ API NAME SIMILARITY	58.33 $\pm$ 0.00	3.78 $\pm$ 2.64	1.71 $\pm$ 0.48	1.14 $\pm$ 0.51
W/O. IN-SITU ADAPTATION	66.67 $\pm$ 0.00	1.77 $\pm$ 1.02	0.76 $\pm$ 0.00	5.91 $\pm$ 3.59
→ FULL RE-GENERATION	26.67 $\pm$ 9.13	89.63 $\pm$ 1.40	2.44 $\pm$ 0.44	91.21 $\pm$ 2.50
RECENT	83.33$\pm$0.00	2.79$\pm$0.21	0.36$\pm$0.11	1.90$\pm$1.84

实验结果： 去掉 Ontology Guidance 或 In-situ adaptation 后，任务成功率（SR）均显著下降，且 Grounding 开销（GO）和等待时间（IT）明显增加。完整的 RECENT 在五项指标上取得最好结果

实验结论： Ontology Guidance 负责找对需要修改的位置并提供正确的替换提示；In-situ adaptation 只修改必要的局部代码，避免完整重生成带来的高开销。两者结合使 RECENT 能够在保持高任务成功率的同时，显著降低开销并保持连续执行。

实验评估

真机实验设置：RECENT 能否部署到真实机器人？



实验目的：验证 RECENT 在真实机器人上的跨 embodiment Skill Grounding 能力，重点考察其是否能够在真实感知与动态障碍物条件下，基于运行时 observation 对未执行代码进行局部 patch。

1. 实验平台与感知配置

Robots

Source Robot: Franka Research 3
(7-DoF, 双指夹爪)

Target Robot: UR7e
(6-DoF, Robotiq 2F-85)



Perception

RGB-D 相机: Intel RealSense D435
Segmentation: SAM 3
Grasp Detection: AnyGrasp



Task

任务: Scan-and-Bag

流程: 抓取商品 → 扫码 → 放入购物篮
每个 trial 共处理 5 件商品



2. 动态障碍物场景设计

执行过程中, scanner 前可能出现 obstacle



可抓取障碍物

通过unit test, 无需修改可以继续执行



不可抓取障碍物

未通过unit test, 无需修改可以继续执行



真机实验结果：RECENT 在真机上是否同样好？

1. 真机实验结果对比

Methods	Success	GC (%↑)	GO (%↓)	EI (count↓)	IT (sec↓)
CaP	4/10	80.00	125.80	3.00	23.51
RECENT	8/10	96.00	15.70	0.40	2.34

实验结果：在真机上部署，RECENT的任务成功率更高、Grounding开销更低、执行连续性更好。

实验结论：RECENT 在真实机器人上同样有效。其只对相关的未来代码进行局部 patch，保留原有 Scan-and-Bag 主逻辑，因此比 CaP 更少重生成、更少中断、等待更短。

提纲

1

研究背景

2

系统设计

3

实验评估

4

总 结

总结

1. 能否进一步提高？

目前RECENT依赖一个预先构建、静态的 Skill Ontology，只要目标机器人需要的能力、API和替代关系已经存在于这个Ontology，就能够完成grounding；但是如果目标机器人中出现了Ontology 中完全没有的新能力或新API，当前方法就无法解决

因此，能不能把Ontology和Skill Repository做成一个动态的，将在部署过程中遇到的新的API映射、能力更新在Ontology和Skill Repository中。

2. 能否泛化？

当前RECENT只能基于Ontology中适配的机器人，超出这个范围就无能为力了。因此，下一步能不能在出现Ontology未覆盖的机器人、新API或者能力时，能自动识别并且把他们加入Ontology，而不依赖人工构建

3. 能否用到我们的场景？

RECENT的兼容性分析不太适用于我们的CaP执行效率优化，但是它的这种“先定位问题，再局部重构”的思想可以借鉴

在我们的场景中，需要进一步研究：如何从CaP代码中定位可以优化的步骤，在不破坏原有任务逻辑的情况下优化、修改代码